

Taller 01 — Foundation Models: Comparación, Decodificación y Razonamiento

MMIA 6013 · IA Generativa y Agentes · Universidad San Francisco de Quito · Informe

Campo	Valor
Estudiante	Miguel Alvarez
Fecha del informe	2026-09-20 (hora de Ecuador, UTC-5)
Repositorio	https://github.com/akroasis0301/taller-01-miguel-alvarez (público)
Fechas de las mediciones	2026-09-19 a 2026-09-21 (UTC); detalle por parte en la tabla de la sección 1
Tarea elegida	Aritmética entera con respuesta verificable automáticamente: 10 casos principales (F1-P1), 3 contaminados (C1-C3) y 3 contaminados difíciles (CD1-CD3); ver el anexo A.
Instrucción exacta (Partes 1 a 4)	Solve the arithmetic problem. Reply with the final integer only: no thousands separators, no decimals, no explanation.
Datos crudos	resultados.csv / resultados.jsonl: 1593 filas, una por llamada, 9 columnas pedidas más fecha, parte, backend, variante, predicción, esperado, costo, error y nota.

Contenido: 1 Resumen y reproducibilidad · 2 Parte 0 · 3 Parte 1 · 4 Parte 2 (2.a y 2.b) · 5 Parte 3 · 6 Parte 4 (4.a y 4.b) · 7 Parte 5 · 8 Presupuesto y costos · 9 Hallazgos y discrepancias · Referencias · Anexo A (casos).

1. Resumen y reproducibilidad

Modelos usados y precios. Los precios salen de la tabla semestral (fuentes/modelos/modelos-2026-1.json), fila por fila, con la fecha de verificación de cada fila; no se tomaron de memoria.

id de la tabla	modelo	proveedor	USD/M entrada	USD/M salida	precio verificado	dónde se usa
openai_razonamiento	gpt-5.6-luna	openai	0.2	1.2	2026-09-18	Partes 1, 2.a y 4
propietario_economico	gpt-4o-mini	openai	0.15	0.6	2026-08-26	Partes 1, 2.a, 2.b y 3
open_weight_pequeno	qwen3:1.7b	ollama	0 (local)	0 (local)	— (cero real)	Partes 1, 2.a y 2.b (top_k)
base_local	openai-community/gpt2	huggingface	0 (local)	0 (local)	— (cero real)	Parte 0

bloque de resultados.csv	filas	con error de API	costo USD
Parte 0.b (local)	10	0	0.000000
Parte 0.c (local)	2	0	0.000000
Parte 1	30	0	0.000582
Parte 1, calentamiento (excluido de tablas)	3	0	0.000055
Parte 2.a	112	6	0.000465
Parte 2.b (API)	750	0	0.006141
Parte 2.b top_k (local)	200	0	0.000000
Parte 3	180	0	0.006902
Parte 4.a (incluye sondas)	186	0	0.009010
Parte 4.b	60	0	0.002395
Parte 4.b extensión	60	0	0.003996
TOTAL	1593	6	0.029546

Las 6 filas con error son rechazos esperados de la Parte 2.a (mensajes literales en la sección 4.1). Las generaciones locales de la Parte 0 y de top_k están en el mismo archivo con costo 0. Todas las tablas se derivan de este archivo.

Fechas de las mediciones por parte

bloque	ejecutado (UTC)	hora de Ecuador (UTC-5)	precio verificado (fila de la tabla)
Parte 0 (0.b y 0.c, GPT-2 local)	2026-09-21 00:23	2026-09-20 19:23	— (local)
Parte 1 (incluye calentamiento)	2026-09-21 00:11-00:12	2026-09-20 19:11-19:12	luna 2026-09-18 · gpt-4o-mini 2026-08-26
Parte 2.a (matriz de exposición)	2026-09-19 16:16-16:39	2026-09-19 11:16-11:39	gpt-4o-mini 2026-08-26 · luna 2026-09-18
Parte 2.b (barrido T × top_p, API)	2026-09-20 01:27-01:28	2026-09-19 20:27-20:28	gpt-4o-mini 2026-08-26
Parte 2.b (top_k, local)	2026-09-20 01:28-01:29	2026-09-19 20:28-20:29	— (local)
Parte 3 (prompting)	2026-09-20 00:52-00:53	2026-09-19 19:52-19:53	gpt-4o-mini 2026-08-26
Parte 4.a (barrido de esfuerzo, con sondas)	2026-09-20 00:54-00:58	2026-09-19 19:54-19:58	luna 2026-09-18
Parte 4.b	2026-09-20 00:58	2026-09-19 19:58	luna 2026-09-18
Parte 4.b extensión	2026-09-20 01:30-01:31	2026-09-19 20:30-20:31	luna 2026-09-18

Las fechas salen de la columna `fecha_utc` de `resultados.csv`. Las Partes 0 y 1 se volvieron a ejecutar el 2026-09-21 00:11-00:23 UTC (2026-09-20 por la tarde en Ecuador), ya en el repositorio final; sus tablas de este informe salen de esa corrida. La fecha de verificación del precio es la de cada fila de la tabla semestral, no la de la ejecución.

Credenciales: ninguna clave aparece en este informe, en los notebooks, en las figuras ni en el repositorio; se leen del archivo `.env`, que `.gitignore` excluye.

2. Parte 0 — El modelo base y la distribución (GPT-2, local, costo 0)

0.a — La distribución del siguiente token

Prefijos: dominio «The result of multiplying 47 by 38 is», seguro «Ladies and» e incierto «After that,».

Cada gráfica muestra $\text{softmax}(\text{logits}/T)$ para los 15 candidatos más probables con $T \in \{0.1, 0.7, 1.0, 1.5, 2.0\}$.

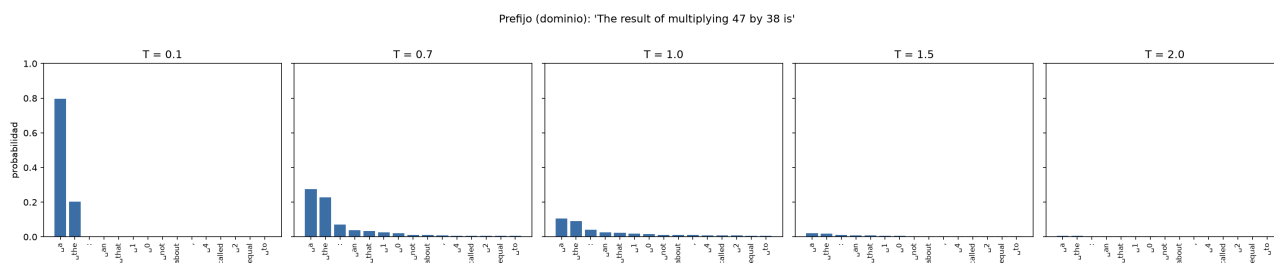


Figura 0.a-1. Prefijo de dominio.

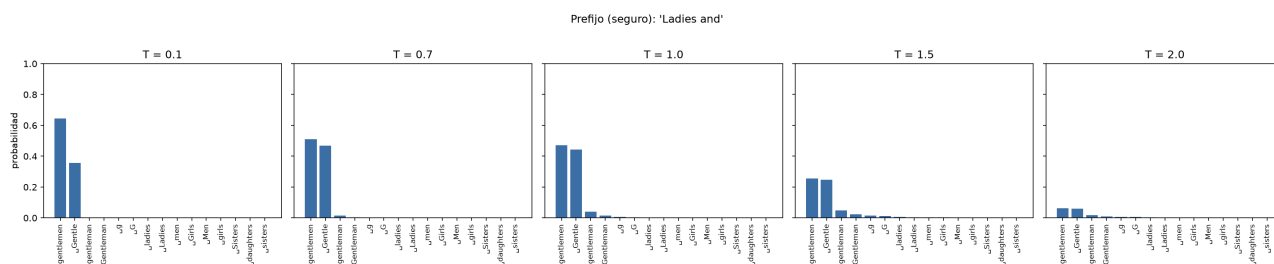


Figura 0.a-2. Prefijo seguro («Ladies and»).

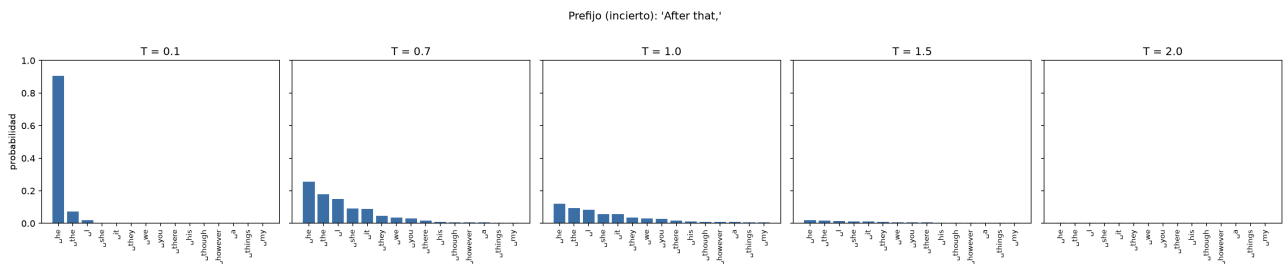


Figura 0.a-3. Prefijo incierto («After that,»).

T	dominio: entropía (bits)	dominio: núcleo 0.9	seguro: entropía (bits)	seguro: núcleo 0.9	incierto: entropía (bits)	incierto: núcleo 0.9
0.1	0.729	2	0.939	2	0.531	1
0.7	4.756	75	1.153	2	3.761	10
1.0	8.158	733	1.678	2	7.892	2957
1.5	11.900	9826	6.973	3668	13.066	16175
2.0	13.656	21016	12.476	16081	14.329	23393

Tabla 0.a. Entropía en bits y número de tokens que acumulan el 90 % de la masa (top_p = 0.9 medido).

Diagnóstico adicional: efecto del token de inicio de documento

prefijo	texto	con BOS	token más probable	p top-1	entropía T=1 (bits)	núcleo 0.9 (T=1)
dominio	The result of multiplying 47 by 38 is	False	«a»	0.1048	8.158	733
dominio	The result of multiplying 47 by 38 is	True	«a»	0.1223	8.406	913
seguro	Ladies and	False	«gentlemen»	0.4714	1.678	2
seguro	Ladies and	True	«gentlemen»	0.6241	1.513	3
incierto	After that,	False	«he»	0.1206	7.892	2957
incierto	After that,	True	«l»	0.1099	7.575	2479
pangrama	The quick brown fox jumps over the lazy	False	«,	0.0557	9.291	2001
pangrama	The quick brown fox jumps over the lazy	True	«,	0.0422	9.418	2013
frase_hecha	Once upon a	False	«time»	0.1766	8.507	1956
frase_hecha	Once upon a	True	«time»	0.9919	0.143	1

Conclusión 0.a

Dividir los logits entre T no cambia el orden de los tokens, solo cuánta masa se lleva cada uno. Con T bajo la distribución se concentra (en el prefijo de dominio, la probabilidad del token más probable pasa de 0.105 con T=1.0 a 0.797 con T=0.1) y con T alto se aplanan hacia la uniforme (la entropía sube de 8.2 a 13.7 bits entre T=1.0 y T=2.0, con un máximo posible de unos 15.6 bits para 50257 tokens). Por eso el tamaño del núcleo del 90 % no es una constante del parámetro: con T=1.0 son 2 tokens tras «Ladies and», 733 tras el prefijo de dominio y 2957 tras «After that,», y en el prefijo seguro el núcleo salta de 2 a 3668 tokens entre T=1.0 y T=1.5. La medición también depende de cómo se presenta el prefijo: sin el token de inicio de documento, «Once upon a» deja la entropía en 8.5 bits, y con él cae a 0.14 bits (la probabilidad de «time» pasa de 0.18 a 0.99), un hallazgo que conviene tener presente al leer cualquier distribución de GPT-2.

0.b — Las tres palancas sobre esa misma distribución

0.b.1 do_sample=False con temperature 0.2 y 1.5 (salida cruda)

temperature=0.2 -> a total of 1,847,847,847,847,847,847,847,847,8

temperature=1.5 -> a total of 1,847,847,847,847,847,847,847,847,8

Con do_sample=False, generate toma el argmax de los logits en cada paso, y la temperatura solo interviene cuando se muestrea. Además, dividir todos los logits entre un T positivo no cambia cuál es el mayor, así que ni en teoría podría alterar el argmax. Transformers lo avisa sin fallar: «The following generation flags are not valid and may be ignored: ['temperature']». Es el estado «aceptado y no actúa»: no hubo error y la salida es idéntica con T=0.2 y con T=1.5 («a total of 1,847,847,847,...»).

0.b.2 top_k=1 con do_sample=True, cinco corridas (semillas 0 a 4), contra do_sample=False

```
greedy (do_sample=False) -> a total of 1,847,847,847,847,847,847,847,847,8
top_k=1, corrida 0 -> a total of 1,847,847,847,847,847,847,847,847,8
top_k=1, corrida 1 -> a total of 1,847,847,847,847,847,847,847,847,8
top_k=1, corrida 2 -> a total of 1,847,847,847,847,847,847,847,847,8
top_k=1, corrida 3 -> a total of 1,847,847,847,847,847,847,847,847,8
top_k=1, corrida 4 -> a total of 1,847,847,847,847,847,847,847,847,8
```

Las cinco corridas con do_sample=True y top_k=1 (semillas 0 a 4) produjeron exactamente los mismos ids que do_sample=False, y el primer token generado (id 257, «`␣a`») coincide con el argmax de la distribución de 0.a. No se infiere de la varianza entre corridas: con k=1 sobrevive un solo token, así que sortear entre un único candidato equivale al argmax.

0.b.3 Dónde corta cada palanca

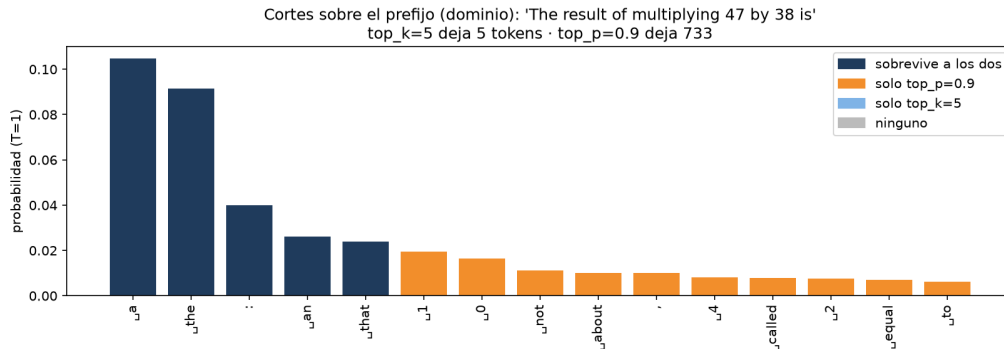


Figura 0.b-1. Cortes sobre el prefijo de dominio.

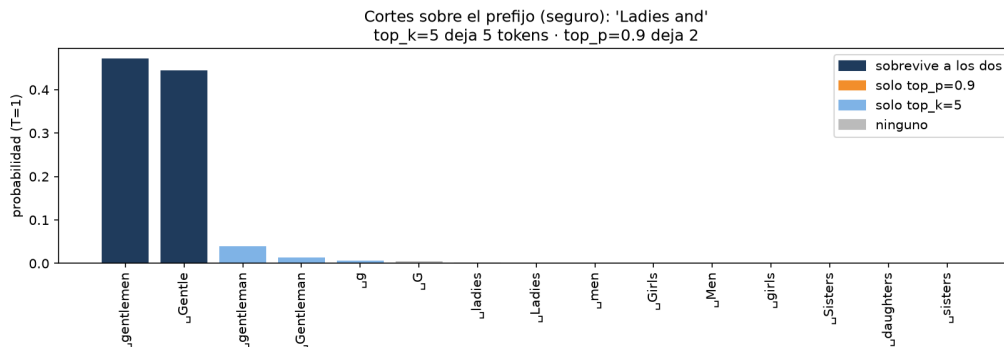


Figura 0.b-2. Cortes sobre el prefijo seguro.

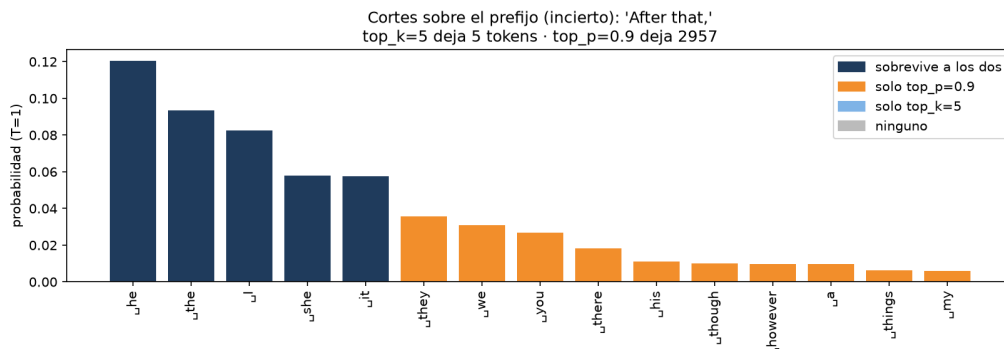


Figura 0.b-3. Cortes sobre el prefijo incierto.

prefijo	tokens con top_k=5	tokens con top_p=0.9	¿coinciden?
dominio	5	733	no
seguro	5	2	no
incierto	5	2957	no

Con top_k=5 sobreviven siempre 5 tokens; con top_p=0.9 sobreviven 733 en el prefijo de dominio, 2 en el seguro y 2957 en el incierto. Los dos conjuntos no coinciden en ninguno de los tres prefijos: en el seguro, k=5 conserva tres tokens que p=0.9 descarta; en el incierto, p=0.9 conserva casi 3000 tokens y k=5 solo cinco. Es la diferencia entre un corte de tamaño fijo y uno de masa fija.

0.b.4 Degeneración: 100 tokens con do_sample=False (salidas crudas)

Prefijo del taller de los unicornios:

The researchers found that the unicorns were able to survive in the wild for up to 10 years, and that they were able to survive in the wild for up to 10,000 years.

"This is a very exciting discovery," said Dr. David H. Hirsch, a professor of ecology and evolutionary biology at the University of California, Berkeley. "It's a very exciting discovery that we can now say that the unicorns are able to survive in the wild for up to

Prefijo de dominio:

[illegible]

Con greedy y el prefijo de dominio la salida es « a total of 1,847,847,847,...», y a los 100 tokens el 93 % de los 4-gramas están repetidos (en el texto de los unicornios, 0.16). Es la degeneración que describen Holtzman et al. (2020): con decodificación de máxima probabilidad el texto se vuelve repetitivo porque la probabilidad de repetir una frase crece con cada repetición.

0.c – El límite del modelo base

Prompt exacto de las Partes 1 a 4 (caso M1) y salida cruda de GPT-2, sin editar. Respuesta esperada: 731.

Solve the arithmetic problem. Reply with the final integer only: no thousands separators, no decimals, no explanation.

Problem: Calculate 43×17

Answer:

Salida cruda, greedy, 100 tokens:

The problem is to solve the problem by multiplying the number of digits by the number of digits.

The problem is to solve the problem by multiplying the number of digits by the number of digits.

The problem is to solve the problem by multiplying the number of digits by the number of digits.

The problem is to solve the problem by multiplying the number of digits by the number of digits.

The problem is to solve the problem by multiplying the number of digits by

Salida cruda, $T=0.7$, 100 tokens:

No, we cannot find the 43×17 problem. We don't need to know the answer to find it. We need to know the answer to find it.

Problem: Solve the arithmetic problem by solving the custom problem. Answer: No, we cannot find the 43×17 problem. We don't need to know the answer to find it. We need to know the answer to find it.

Problem: Solve the custom problem by solving the custom problem. Answer:

GPT-2 no obedece: trata la instrucción como texto que hay que continuar. En la salida pegada arriba repite «The problem is to solve the problem by multiplying the number of digits by the number of digits» una y otra vez y nunca escribe 731; con T=0.7 inventa otro «Problem:» y sigue. No hay tarea para él, solo un prefijo cuya continuación más probable en su corpus es más texto parecido. Lo que les falta son los pasos posteriores al preentrenamiento: el ajuste por instrucciones, con pares petición-respuesta y una plantilla de chat que marca dónde termina la petición, y la alineación con preferencias humanas, que enseñan «si alguien pide algo, se responde y se para». gpt-5.6-luna, gpt-4o-mini y qwen3:1.7b pasaron por ese entrenamiento; por eso, con el mismo prompt, sí devuelven un entero, aunque no siempre respeten el formato.

3. Parte 1 — Comparación de modelos

Mismo prompt sobre tres modelos de la tabla semestral, una corrida por caso, secuencial. Condiciones: gpt-4o-mini y qwen3:1.7b con temperature=0; gpt-5.6-luna no admite temperature con su esfuerzo por defecto (sección 4.1), así que va con effort=medium y un tope de 16000 tokens de salida. Latencia = tiempo de pared de UNA petición completa, sin streaming; no incluye la carga del modelo local (la absorbe la llamada de calentamiento).

Ranuras de la comparación. Propietario grande: openai_razonamiento (gpt-5.6-luna), que la nota del enunciado admite para esa ranura con la clave del curso. Propietario económico: propietario_economico (gpt-4o-mini). Open-weight local: open_weight_pequeno (qwen3:1.7b servido por Ollama). No hay clave de Anthropic, así que no se usaron sus filas. Los precios se leyeron de fuentes/modelos/modelos-2026-1.json, no de memoria, y cada uno lleva la fecha de verificación de su fila (luna 2026-09-18, gpt-4o-mini 2026-08-26).

métrica	openai_razonamiento	propietario_economico	open_weight_pequeno
modelo	gpt-5.6-luna	gpt-4o-mini	qwen3:1.7b
exactitud sobre 10 casos	10/10	3/10	4/10
formato correcto (%)	100	100	20
latencia media (s)	1.401	1.083	0.285
latencia mediana (s)	1.405	0.911	0.294
tokens de entrada (media)	40.7	41.7	55.5
tokens de salida (media)	35.1	2.8	21.8
tokens de razonamiento (media)	27.2	0.0	0.0
costo 10 casos (USD)	0.000503	0.000079	0
costo por 1000 problemas (USD)	0.0503	0.0079	0
USD/M entrada	0.2	0.15	0
USD/M salida	1.2	0.6	0
precio verificado el	2026-09-18	2026-08-26	- (cero real)

Tabla 1. Derivada de resultados.csv (parte 1). Precios leídos de la tabla semestral, con la fecha de cada fila.

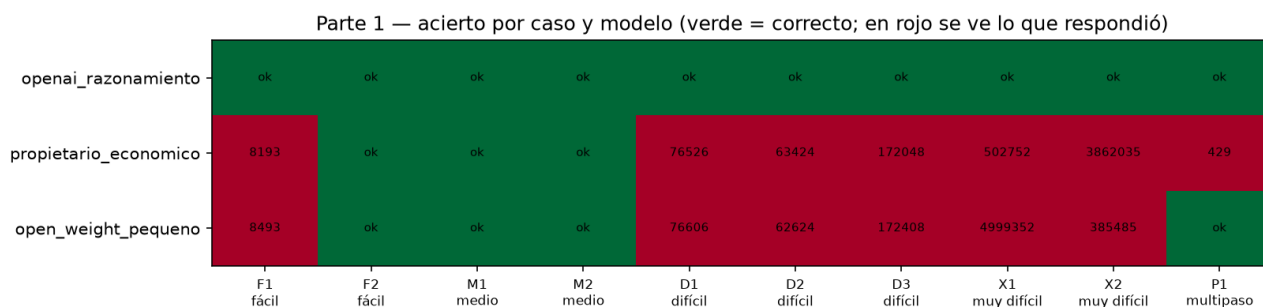


Figura 1. Aciertos por caso y modelo.

Errores por caso (respuestas incorrectas)

modelo	caso	esperado	predicción	error relativo (%)	orden de magnitud ok
propietario_economico	F1	8393	8193	2.3829	True
propietario_economico	D1	76626	76526	0.1305	True
propietario_economico	D2	63344	63424	0.1263	True
propietario_economico	D3	172568	172048	0.3013	True
propietario_economico	X1	5021952	502752	89.9889	False
propietario_economico	X2	3863735	3862035	0.044	True
propietario_economico	P1	419	429	2.3866	True
open_weight_pequeno	F1	8393	8493	1.1915	True
open_weight_pequeno	D1	76626	76606	0.0261	True
open_weight_pequeno	D2	63344	62624	1.1367	True
open_weight_pequeno	D3	172568	172408	0.0927	True
open_weight_pequeno	X1	5021952	4999352	0.45	True
open_weight_pequeno	X2	3863735	385485	90.023	False

Observaciones

Calidad. gpt-5.6-luna acertó 10 de 10, gpt-4o-mini 3 de 10 y qwen3:1.7b 4 de 10 con un extractor que toma el último número de la salida; solo 2 de las 10 salidas de qwen son un entero limpio.

¿Alucina? Ningún modelo se abstiene: los errores son números plausibles. gpt-4o-mini se equivoca entre 0.04 % y 2.4 % del valor esperado, salvo en X1 (502752 en vez de 5021952: perdió un dígito, error del 90 %). qwen3:1.7b comete el mismo tipo de error en X2 (385485 en vez de 3863735).

Formato y verbosidad. luna y gpt-4o-mini devuelven solo un entero en el 100 % de los casos (2.8 tokens de salida el segundo). qwen3:1.7b escribe la ecuación o «Final answer:» en 8 de 10 casos (21.8 tokens de salida). luna factura 35.1 tokens de salida por llamada, de los cuales 27.2 son de razonamiento que no se ve.

Costo y latencia. Con los precios de la tabla (luna 0.20/1.20 USD por millón, verificado 2026-09-18; gpt-4o-mini 0.15/0.60, verificado 2026-08-26) resultan 0.0503 y 0.0079 USD por 1000 problemas; qwen3:1.7b es local y cuesta 0. Latencia media: 1.401 s (luna), 1.083 s (gpt-4o-mini) y 0.285 s (qwen local); es el tiempo de pared de una petición completa sin streaming, no comparable en sentido estricto entre API y local.

Supuesto contra medición. Estimé 3000 tokens de salida por llamada para luna (0.0362 USD para la parte) y medí 35.1: el gasto real de la Parte 1 fue de 0.0006 USD, unas 57 veces menos.

Conclusión: ¿qué modelo elegiría en producción?

Elegiría gpt-5.6-luna. Es el único de los tres que resuelve la tarea (10 de 10 frente a 3 y 4 de 10), y aunque cuesta unas 6 veces más por 1000 problemas que gpt-4o-mini (0.0503 frente a 0.0079 USD), sigue siendo céntimos, y su latencia de 1.4 s es aceptable salvo en interfaces muy interactivas. El costo por respuesta correcta engaña: gpt-4o-mini sale más barato (0.000026 frente a 0.000050 USD) pero entrega 7 respuestas equivocadas de cada 10 con aspecto de correctas, y eso exigiría un verificador aparte. qwen3:1.7b queda para casos de costo cero o datos que no pueden salir de la máquina, siempre con la salida validada, porque solo respeta el formato en el 20 % de los casos. Con una corrida y 10 casos la diferencia entre modelos es clara pero el detalle es frágil; y para aritmética real en producción lo correcto sería una calculadora como herramienta, no un modelo.

4. Parte 2 — Decodificación

4.1 — 2.a: matriz de exposición (prueba del 2026-09-19)

modelo	parámetro	lo que declara la tabla	lo que observé	fecha de la prueba	¿coincide?
gpt-4o-mini	temperature	si	aceptado y actúa	2026-09-19	sí
gpt-4o-mini	top_p	si	aceptado y actúa	2026-09-19	sí
gpt-4o-mini	top_k	no	rechazado	2026-09-19	sí
gpt-5.6-luna	temperature	no_en_esta_fila	effort=medium(defecto): solo_valor_por_defecto effort=none: aceptado y actúa	2026-09-19	NO — discrepancia
gpt-5.6-luna	top_p	no_en_esta_fila	effort=medium(defecto): rechazado effort=none: aceptado y actúa	2026-09-19	NO — discrepancia
gpt-5.6-luna	top_k	no	effort=medium(defecto): rechazado effort=none: rechazado	2026-09-19	sí
qwen3:1.7b	temperature	si	aceptado y actúa	2026-09-19	sí
qwen3:1.7b	top_p	si	aceptado y actúa	2026-09-19	sí
qwen3:1.7b	top_k	si	aceptado y actúa	2026-09-19	sí

Tabla 2.a. Estados: «aceptado y actúa» = 200 y salida distinta entre valores extremos; «rechazado» = error con el mensaje literal de abajo. Regla fijada de antemano: 8 corridas por condición.

Mensajes de error literales de las celdas rechazadas

gpt-4o-mini · top_k · 2026-09-19

```
[defecto] BadRequestError | codigo=400 | mensaje=Error code: 400 - {'error': {'message': "Unknown parameter: 'top_k'. Did you mean 'top_p'?", 'type': 'invalid_request_error', 'param': 'top_k', 'code': 'unknown_parameter'}} | cuerpo={'message': "Unknown parameter: 'top_k'. Did you mean 'top_p'?", 'type': 'invalid_request_error', 'param': 'top_k', 'code': 'unknown_parameter'}
```

gpt-5.6-luna · temperature · 2026-09-19

```
[effort=medium(defecto)] BadRequestError | codigo=400 | mensaje=Error code: 400 - {'error': {'message': "Unsupported parameter: 'temperature' is not supported with this model.", 'type': 'invalid_request_error', 'param': 'temperature', 'code': None}} | cuerpo={'message': "Unsupported parameter: 'temperature' is not supported with this model.", 'type': 'invalid_request_error', 'param': 'temperature', 'code': None}
```

gpt-5.6-luna · top_p · 2026-09-19

```
[effort=medium(defecto)] BadRequestError | codigo=400 | mensaje=Error code: 400 - {'error': {'message': "Unsupported parameter: 'top_p' is not supported with this model.", 'type': 'invalid_request_error', 'param': 'top_p', 'code': None}} | cuerpo={'message': "Unsupported parameter: 'top_p' is not supported with this model.", 'type': 'invalid_request_error', 'param': 'top_p', 'code': None}
```

gpt-5.6-luna · top_k · 2026-09-19

```
[effort=medium(defecto)] BadRequestError | codigo=400 | mensaje=Error code: 400 - {'error': {'message': "Unknown parameter: 'top_k'. Did you mean 'top_p'?", 'type': 'invalid_request_error', 'param': 'top_k', 'code': 'unknown_parameter'}} | cuerpo={'message': "Unknown parameter: 'top_k'. Did you mean 'top_p'?", 'type': 'invalid_request_error', 'param': 'top_k', 'code': 'unknown_parameter'}} || [effort=none] BadRequestError | codigo=400 | mensaje=Error code: 400 - {'error': {'message': "Unknown parameter: 'top_k'. Did you mean 'top_p'?", 'type': 'invalid_request_error', 'param': 'top_k', 'code': 'unknown_parameter'}} | cuerpo={'message': "Unknown parameter: 'top_k'. Did you mean 'top_p'?", 'type': 'invalid_request_error', 'param': 'top_k', 'code': 'unknown_parameter'}}
```

Evidencia: respuestas distintas por condición (8 corridas)

modelo	régimen	condición	corridas	con error	distintas	respuestas
open_weight_pequeno	think=False	BASE	8	0	8	eagle, fox, sloth, cow, bird, zebra, crocodile, turtle
open_weight_pequeno	think=False	CTRL_tipo	8	0	1	tiger×8
open_weight_pequeno	think=False	T0	8	0	1	tiger×8
open_weight_pequeno	think=False	TOPK1	8	0	1	tiger×8
open_weight_pequeno	think=False	TOPP_BAJO	8	0	1	tiger×8
openai_razonamiento	effort=none	BASE	8	0	5	capybara×3, axolotl×2, penguin, wombat, quokka
openai_razonamiento	effort=none	T0	8	0	1	quokka×8
openai_razonamiento	effort=none	TOPP_BAJO	8	0	1	quokka×8
propietario_economico	defecto	BASE	8	0	7	kangaroo×2, cheetah, quokka, octopus, giraffe, ocelot, aardvark
propietario_economico	defecto	T0	8	0	1	octopus×8
propietario_economico	defecto	TOPP_BAJO	8	0	1	octopus×8

BASE = sin restricción (temperature 1.6); T0 = temperature 0; TOPP_BAJO = top_p 0.01 con temperature 1.6; TOPK1 = top_k 1 con temperature 1.6; CTRL_tipo = «temperature».

Observaciones y conclusiones de 2.a

Prueba realizada el 2026-09-19, 8 corridas por condición y un solo prompt. Los mensajes de error literales están en la tabla.

gpt-4o-mini (propietario_economico) coincide con la tabla en las tres celdas. temperature y top_p son «aceptado y actúa» (con temperature=0 salió 1 respuesta distinta de 8; con 1.6, 7). top_k es «rechazado» con 400 unknown_parameter, coherente con que OpenAI no expone top_k.

gpt-5.6-luna (openai_razonamiento): hallazgo. La tabla declara no_en_esta_fila y eso solo es cierto en parte. Con el esfuerzo por defecto (medium), temperature=1.6 devuelve 400 «Unsupported parameter: 'temperature' is not supported with this model.» y top_p devuelve el mismo error con 0.01 y con su valor por defecto 1.0; solo acepta temperature=1.0, que es su valor por defecto. Con effort=none, temperature y top_p se aceptan y actúan (temperature=0: 1 respuesta distinta; temperature=1.6: 5). El estado de la celda depende de un tercer parámetro, effort. Corrección propuesta para la tabla del curso: temperature y top_p se rechazan salvo con reasoning.effort=none.

qwen3:1.7b (Ollama) coincide con la tabla: los tres diales son «aceptado y actúan» (1 respuesta distinta con temperature=0, top_p=0.01 y top_k=1, contra 8 con temperature=1.6).

Controles: un 200 no prueba que el parámetro exista. Ollama acepta con 200 temperature=-5 y una clave inventada (top_kk=1). Con temperature=1.6 salieron 8 respuestas distintas sin restricción, 1 con top_k=1 bien escrito y 6 con top_kk=1: la clave inventada no hizo nada. El control con «temperature» no discrimina, porque equivale a no mandar temperatura y el valor por defecto del modelo (temperature 0.6, top_k 20, top_p 0.95, según /api/show) ya colapsa a una sola respuesta. Consecuencia: en local hay que fijar temperature, top_p y top_k de forma explícita en cada llamada.

Lo que no se pudo comprobar: no hay clave de Anthropic, así que la única fila sin_verificar de la tabla (propietario_balanceado) queda sin resolver. Se resuelve solo mandando la petición a esa API.

4.2 — 2.b: barrido de temperature × top_p (gpt-4o-mini, 750 llamadas) y de top_k (local)

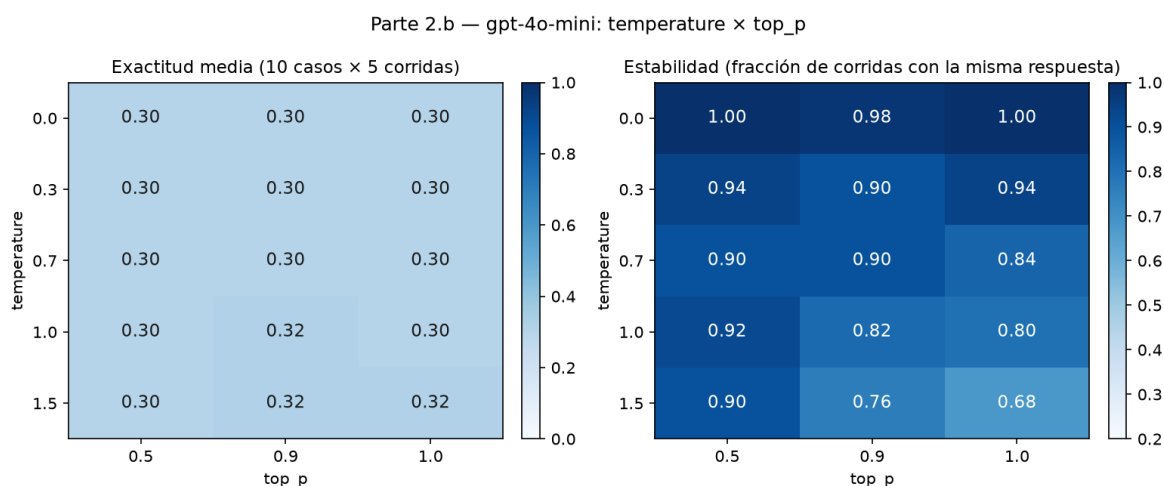


Figura 2.b. Exactitud y estabilidad por celda (5 corridas $\times$ 10 casos).

T	top_p	llamadas ok	exactitud	sd entre corridas	estabilidad	casos 5 de 5	tokens de salida (media)	costo (USD)
0.0	0.5	50	0.30	0.000	1.00	10	2.80	0.000397
0.0	0.9	50	0.30	0.000	0.98	9	2.80	0.000397
0.0	1.0	50	0.30	0.000	1.00	10	2.80	0.000397
0.3	0.5	50	0.30	0.000	0.94	8	2.80	0.000397
0.3	0.9	50	0.30	0.000	0.90	8	2.80	0.000397
0.3	1.0	50	0.30	0.000	0.94	7	2.80	0.000397
0.7	0.5	50	0.30	0.000	0.90	7	2.82	0.000397
0.7	0.9	50	0.30	0.000	0.90	8	2.80	0.000397
0.7	1.0	50	0.30	0.000	0.84	7	2.84	0.000398
1.0	0.5	50	0.30	0.000	0.92	8	2.84	0.000398
1.0	0.9	50	0.32	0.045	0.82	5	2.90	0.000400
1.0	1.0	50	0.30	0.000	0.80	6	2.88	0.000399
1.5	0.5	50	0.30	0.000	0.90	7	2.86	0.000399
1.5	0.9	50	0.32	0.045	0.76	5	2.90	0.000400
1.5	1.0	50	0.32	0.045	0.68	5	8.70	0.000574

Tabla 2.b. Estabilidad = fracción de las 5 corridas de un caso que dan la respuesta más frecuente, promediada sobre los 10 casos. qpt-40-mini, 0.15/0.60 USD por millón (verificado 2026-08-26); ejecutada el 2026-09-20 (UTC).

top k en local (qwen3:1.7b, temperature=1.0 explícita, 5 corridas x 10 casos)

variante	llamadas ok	exactitud	estabilidad de la respuesta	casos 5 de 5 (respuesta)	casos 5 de 5 (texto)	tokens de salida
top_k=1	50	0.40	1.00	10	10	21.70
top_k=5	50	0.42	0.72	5	1	26.76
top_k=40	50	0.48	0.68	4	1	28.48
greedy(T=0)	50	0.40	1.00	10	10	21.70

top k=1 reproduce greedy: las mismas 5 salidas en los 10 casos y idénticas a T=0 (10 de 10).

De la distribución (0.b) a la exactitud (2.b)

En 0.b top_k=1 se comprobó contra la distribución: los mismos ids que greedy sobre un prefijo, sin inferir nada de la varianza. Aquí cambia la unidad de medida: ya no se mira qué tokens sobreviven al corte, sino cuántos de los 10 problemas salen bien y cuán estable es cada respuesta. Con top_k=1 el resultado vuelve a ser el de greedy (10 de 10 casos idénticos, exactitud 0.40, estabilidad 1.00). Con top_k=5 y 40 la exactitud (0.42 y 0.48) queda dentro del ruido, porque 0.08 son 4 de 50 llamadas sobre solo 10 problemas distintos, y la estabilidad baja a 0.72 y 0.68: un corte más ancho deja pasar más variabilidad sin una mejora demostrable en aciertos.

T = 0: ¿las corridas coinciden?

caso	salidas con T=0	distintas	detalle
F1	15	1	8193×15
F2	15	1	311×15
M1	15	1	731×15
M2	15	1	2160×15
D1	15	1	76526×15
D2	15	1	63424×15
D3	15	2	172048×14, 172488×1
X1	15	1	502752×15
X2	15	1	3862035×15
P1	15	1	429×15

9 de 10 casos idénticos en las 15 corridas. D3 dio 172488 una vez (con top_p=0.9): T=0 no es una promesa de reproducibilidad.

Conclusión de 2.b

Para extraer un campo o resolver una cuenta elegiría temperature=0 con top_p=1.0: estabilidad 1.00, 10 de 10 casos con las 5 corridas idénticas y la misma exactitud (0.30) que cualquier otra celda. Subir T no compró exactitud: el rango 0.30–0.32 es, como mucho, una llamada de 50 y solo hay 10 problemas distintos. Lo que sí cuesta es estabilidad, que baja a 0.68 con T=1.5 y top_p=1.0, y en esa celda aparecieron 3 salidas de 100 tokens de texto sin sentido (D2, longitud media 8.7 frente a 2.8). La temperatura divide los logits antes del softmax: con T bajo concentra la masa en pocos tokens y con T alto la reparte entre muchos más, de modo que la salida pasa de ser casi una medición a ser una muestra de una distribución cada vez más ancha. Además T=0 no es una promesa: en D3 con top_p=0.9 una corrida de 5 dio 172488 en vez de 172048. Para tres titulares alternativos elegiría T=1.0 con top_p=0.9: buscaría variedad entre corridas (estabilidad 0.82) sin llegar a la zona degenerada; es un juicio apoyado en estabilidad y longitud, porque aquí no se midió calidad de titulares. En local, top_k=1 reproduce greedy en 10 de 10 casos; top_k=5 y 40 bajan la estabilidad a 0.72 y 0.68.

5. Parte 3 — Prompting estructurado (gpt-4o-mini, T=0, 10 casos × 3 corridas)

Variantes: 1 zero-shot; 2 few-shot (3 ejemplos); 3 Chain-of-Thought; 4a JSON pedido en el prompt; 4b modo JSON (json_object); 4c esquema estricto (json_schema, strict). Tokens de entrada y de salida por separado.

variante	llamadas	exactitud	exactitud %	tokens entrada	tokens salida	USD por 1000 problemas	USD por respuesta correcta
1_zero-shot	30	9/30	30.0	41.7	2.8	0.0079	0.0000264
2_few-shot	30	9/30	30.0	95.7	2.8	0.016	0.0000534
3_CoT	30	27/30	90.0	54.7	272.2	0.1715	0.0001906
4a_JSON-en-prompt	30	9/30	30.0	39.7	7.7	0.0106	0.0000353
4b_modos-JSON	30	9/30	30.0	39.7	7.7	0.0106	0.0000353
4c_esquema-estricto	30	9/30	30.0	61.7	6.9	0.0134	0.0000446

Tabla 3. Derivada de resultados.csv (parte 3). Precio: 0.15/0.60 USD por millón, verificado el 2026-08-26.

caso	1_zero-shot	2_few-shot	3_CoT	4a_JSON-en-prompt	4b_modulo-JSON	4c_esquema-estricto
F1	0/3	0/3	3/3	0/3	0/3	0/3
F2	3/3	3/3	3/3	3/3	3/3	3/3
M1	3/3	3/3	3/3	3/3	3/3	3/3
M2	3/3	3/3	3/3	3/3	3/3	3/3
D1	0/3	0/3	3/3	0/3	0/3	0/3
D2	0/3	0/3	3/3	0/3	0/3	0/3
D3	0/3	0/3	3/3	0/3	0/3	0/3
X1	0/3	0/3	0/3	0/3	0/3	0/3
X2	0/3	0/3	3/3	0/3	0/3	0/3
P1	0/3	0/3	3/3	0/3	0/3	0/3

Aciertos por caso y variante (de 3 corridas).

Salida estructurada: garantía del proveedor contra verificación propia

modo	lo que el proveedor garantiza	salidas	parsean tal cual	parsean tras quitar cercas	cumplen el esquema	valor correcto
4a_JSON-en-prompt	nada: es una petición en el prompt	30	30	30	30	9
4b_modulo-JSON	que la salida parsea como JSON	30	30	30	30	9
4c_esquema-estricto	que parsea y cumple claves y tipos del esquema	30	30	30	30	9

Ninguno de los tres modos garantiza que el valor sea el correcto; esa comprobación es propia (columna «valor correcto»).

Respuesta completa de un caso (D3 = 583 × 296, esperado 172568; corrida 0)

1_zero-shot · acierto=0 · tokens entrada/salida: 40/3

172048

2_few-shot · acierto=0 · tokens entrada/salida: 94/3

172448

3_CoT · acierto=1 · tokens entrada/salida: 53/223

To solve the problem (583×296) , we can break it down step by step.

- Break down the multiplication**:

$$583 \times 296 = 583 \times (300 - 4) = 583 \times 300 - 583 \times 4$$
- Calculate (583×300)** :

$$583 \times 300 = 583 \times 3 \times 100 = 1749 \times 100 = 174900$$
- Calculate (583×4)** :

$$583 \times 4 = 2332$$
- Combine the results**:

$$583 \times 296 = 174900 - 2332$$
- Perform the subtraction**:

$$174900 - 2332 = 172568$$

Thus, the final answer is:

Answer: 172568

4a_JSON-en-prompt · acierto=0 · tokens entrada/salida: 38/8

{"answer": 172488}

4b_modulo-JSON · acierto=0 · tokens entrada/salida: 38/8

{"answer": 172488}

```
{"answer":172488}
```

Conclusión de la Parte 3

Con gpt-4o-mini a T=0, 10 casos y 3 corridas por variante: zero-shot acierta 9 de 30 y cuesta 0.0079 USD por 1000 problemas; lo usaría cuando el modelo ya sabe la tarea y solo hace falta la línea base. Few-shot también acierta 9 de 30 y cuesta el doble (0.016) por los 54 tokens de entrada extra: sirve para fijar formato o estilo, no para enseñar aritmética. Chain-of-Thought sube a 27 de 30 (90 %), pero con 272 tokens de salida frente a 2.8 cuesta 22 veces más por problema y 7 veces más por respuesta correcta (0.00019 frente a 0.000026 USD): vale cuando el error cuesta más que eso y el problema necesita pasos intermedios (F1, D1, D2, D3, X2 y P1 pasaron de 0 a 3 de 3; X1 siguió en 0 de 3), y no vale donde el zero-shot ya acierta (F2, M1 y M2 salieron 3 de 3 sin razonar). La salida estructurada no cambió la exactitud (9 de 30 en las tres), pero separa dos cosas: lo que el proveedor garantiza y lo que verificamos. Pedir JSON en el prompt no garantiza nada, el modo JSON garantiza que parsea y el esquema estricto garantiza además claves y tipos; ninguno garantiza el valor. Verificamos que las 30 salidas de cada modo parseen y cumplen el esquema, y que solo 9 tienen el valor correcto. El esquema estricto añade 22 tokens de entrada (61.7 frente a 39.7): es un seguro barato, y la verificación del valor sigue siendo nuestra.

Corrección del verificador: la primera versión de mi extractor de CoT no reconocía «\text{Answer: } N» y contó como error 8 respuestas correctas (19 de 30 en vez de 27). Se corrigió y se reevaluaron las salidas guardadas sin llamar de nuevo a la API; las 8 filas quedaron marcadas «reevaluado» en la columna nota de resultados.csv.

6. Parte 4 — Modelos de razonamiento (gpt-5.6-luna)

Dial: reasoning.effort (Responses API), niveles none, low, medium, high, xhigh y max; por defecto medium. Contador: usage.output_tokens_details.reasoning_tokens. Precio: 0.20/1.20 USD por millón, verificado el 2026-09-18. Sonda previa de los 6 niveles: los seis fueron aceptados. 4.a: 10 casos × 3 corridas × 6 niveles = 180 llamadas.

Regla de presupuesto («un nivel completo antes de los tres»): antes del barrido corrí el nivel más alto, max, sobre los 10 casos (1 corrida, 10 llamadas). Salió razonamiento medio 40 tokens, salida total media 48 y 0.000065 USD por llamada, contra los 16000 tokens de salida que suponía el enunciado para ese nivel. Con eso rehice la cuenta (cota de 0.012 USD para las 180 llamadas, muy por debajo del freno de 0.60 USD) y seguí con el barrido.

4.a — Barrido de esfuerzo

nivel	exactitud	IC95 % (Wilson)	razonamiento o medio (sd)	rango de razonamiento	visibles (media)	latencia media (s)	USD por 1000 problemas	USD por correcta	truncadas
none	27/30 = 90.0 %	74-97	0.0 (0.0)	0-0	5.9	1.01	0.01522	0.000017	0
low	30/30 = 100.0 %	89-100	25.4 (10.2)	0-44	7.8	1.28	0.0479	0.000048	0
medium	30/30 = 100.0 %	89-100	27.9 (8.3)	14-44	7.9	1.15	0.0511	0.000051	0
high	30/30 = 100.0 %	89-100	29.8 (9.5)	17-56	7.9	1.14	0.05342	0.000053	0
xhigh	30/30 = 100.0 %	89-100	33.5 (14.3)	18-81	7.9	1.45	0.05782	0.000058	0
max	30/30 = 100.0 %	89-100	40.2 (22.4)	22-117	7.9	1.27	0.06582	0.000066	0

Tabla 4.a. Tokens de salida visibles = total de salida menos razonamiento. Costo con el razonamiento incluido, al precio de la fila.

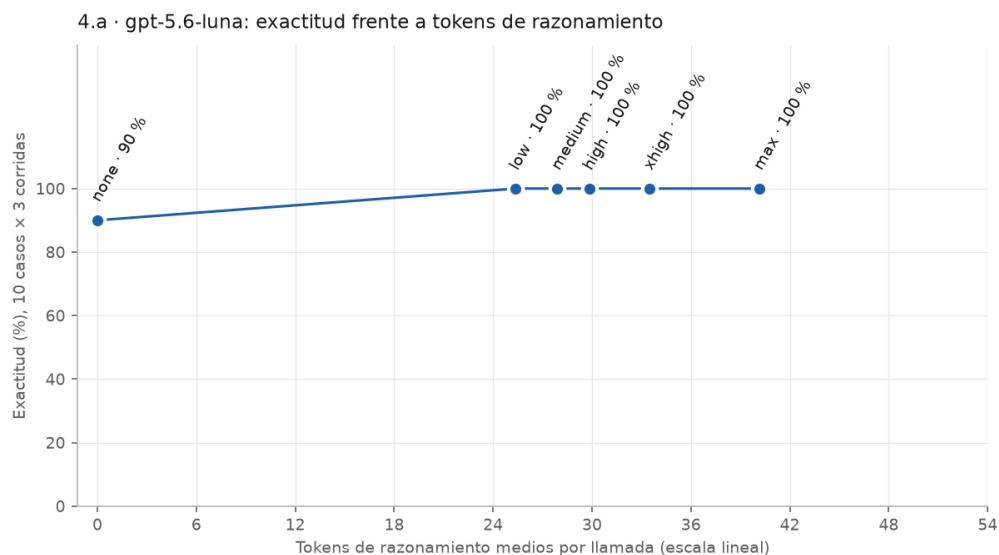


Figura 4.a-1. Exactitud frente a tokens de razonamiento (escala lineal).

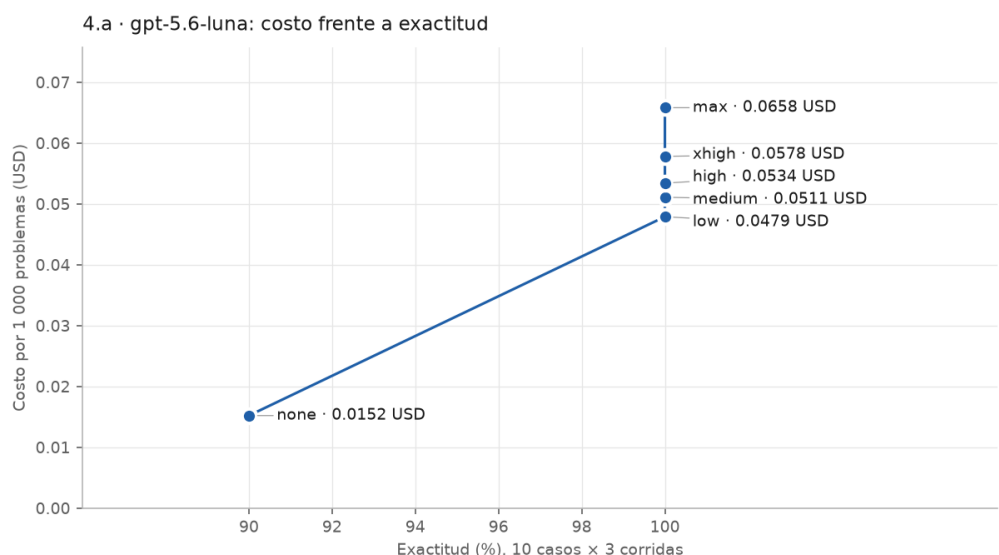


Figura 4.a-2. Costo por 1000 problemas frente a exactitud.

Diagnóstico del dial: se mira el contador, no la prosa

nivel	razonamiento medio	veredicto por promedio (umbral: +10 tokens o x1.25)	detalle		
none	0.0	línea base	OK: none apaga el razonamiento		
low	25.4	aceptado y actúa	+25.4 tokens respecto del anterior		
medium	27.9	aceptado y NO actúa (no se distingue del nivel anterior)	x1.10 respecto del anterior		
high	29.8	aceptado y NO actúa (no se distingue del nivel anterior)	x1.07 respecto del anterior		
xhigh	33.5	aceptado y NO actúa (no se distingue del nivel anterior)	x1.12 respecto del anterior		
max	40.2	aceptado y NO actúa (no se distingue del nivel anterior)	x1.20 respecto del anterior		
comparación pareada por caso	casos con más razonamiento	empates	casos con menos	diferencia media (tokens)	p (prueba de signos)
low frente a none	10	0	0	25.4	0.0020
medium frente a low	8	2	0	2.5	0.0078
high frente a medium	7	2	1	1.9	0.0703
xhigh frente a high	9	1	0	3.7	0.0039
max frente a xhigh	10	0	0	6.7	0.0020
max frente a low	10	0	0	14.8	0.0020

Dos lentes: el promedio con umbrales fijados de antemano no distingue medium de low, pero la comparación pareada por caso muestra que el contador sube en todos los pasos. Se reportan ambas: el dial actúa, con poca amplitud.

Tokens de razonamiento medios por caso y nivel

caso	none	low	medium	high	xhigh	max
F1	0.0	24.0	24.0	26.7	28.7	30.3
F2	0.0	22.0	22.0	23.0	24.3	28.3
M1	0.0	22.7	26.7	23.7	27.3	32.0
M2	0.0	5.7	15.3	18.0	18.0	22.0
D1	0.0	27.0	29.0	29.0	30.7	35.3
D2	0.0	28.0	31.3	31.3	34.3	37.0
D3	0.0	25.3	26.7	31.7	34.7	37.3
X1	0.0	36.0	39.0	41.7	45.7	48.7
X2	0.0	43.3	44.0	50.0	67.7	102.3
P1	0.0	19.7	21.0	23.3	23.7	28.3

Costo marginal

nivel	delta exactitud (pp)	delta USD por 1000	USD por punto por 1000	lectura
none	-	-	-	línea base
low	10.0	0.0327	0.00327	+10.0 pp por +0.0327 USD/1000
medium	0.0	0.0032	-	0 pp de mejora por +0.0032 USD/1000
high	0.0	0.0023	-	0 pp de mejora por +0.0023 USD/1000
xhigh	0.0	0.0044	-	0 pp de mejora por +0.0044 USD/1000
max	0.0	0.008	-	0 pp de mejora por +0.0080 USD/1000

Conclusión de 4.a

Mi punto de rendimientos decrecientes es el primer escalón: de none a low la exactitud sube de 90 % a 100 % (27 a 30 aciertos de 30; el único caso que falla sin razonamiento es X1) por 0.0327 USD más por 1000 problemas, es decir 0.00327 USD por punto porcentual. A partir de low no hay puntos que comprar: medium, high, xhigh y max cuestan 0.0032, 0.0055, 0.0099 y 0.0179 USD más por 1000 problemas que low y aportan 0 puntos; max cuesta 1.4 veces más que low. La curva es plana desde low, y ese también es un resultado. El dial sí actúa: el contador sube en cada nivel (25.4, 27.9, 29.8, 33.5 y 40.2 tokens) y en la comparación pareada por caso, max razona más que low en 10 de 10 casos ($p=0.002$). Pero lo hace con muy poca amplitud, unos 15 tokens entre low y max, frente a los 1500-16000 que suponía el presupuesto del taller (lo medido es entre 0.3 % y 2.2 % de lo supuesto). Con problemas de esta dificultad y una exactitud ya en el techo, pensar más no tiene qué mejorar; el dial se mueve más en el caso más difícil (X2: de 43 a 102 tokens). Límite: 10 casos, intervalo de Wilson 89-100 % para los niveles perfectos.

4.b — El caso donde pensar más hace daño

Tres casos contaminados en los niveles más bajo (none) y más alto (max) de esfuerzo, 10 corridas por caso: C1 «Marta tiene 5 manzanas...» con información irrelevante (esperado 8); C2 «Casi todos responden 24...» con un marco engañoso, 6×7 (esperado 42); C3 «En los últimos cinco ejercicios la respuesta fue par y terminó en 0...» con una correlación espuria, 13 + 14 (esperado 27).

nivel	caso	aciertos	exactitud %	razonamiento medio	visibles (media)	respuestas dadas	truncadas
none	C1	10/10	100.0	0.0	5.0	{8: 10}	0
none	C2	10/10	100.0	0.0	5.0	{42: 10}	0
none	C3	10/10	100.0	0.0	5.0	{27: 10}	0
none	TOTAL	30/30	100.0	0.0	5.0	IC95 de Wilson: 89-100 %	0
max	C1	10/10	100.0	23.7	7.0	{8: 10}	0
max	C2	10/10	100.0	37.7	7.0	{42: 10}	0
max	C3	10/10	100.0	45.2	7.0	{27: 10}	0
max	TOTAL	30/30	100.0	35.5	7.0	IC95 de Wilson: 89-100 %	0

Extensión: contaminados con núcleo difícil (CD1-CD3)

nivel	caso	contaminación	núcleo	contaminado	sin contaminar (4.a)	razon. contaminado	razon. sin contaminar	lectura	respuestas dadas
none	CD1	marco engañoso	X1	10/10	0/3	0.0	0.0	no informa: ya falla sin contaminar	{5021952: 10}
none	CD2	información irrelevante	X2	8/10	3/3	0.0	0.0		{3863735: 8, 3863835: 1, 3864035: 1}
none	CD3	correlación espuria	D2	10/10	3/3	0.0	0.0		{63344: 10}
max	CD1	marco engañoso	X1	10/10	3/3	95.5	48.7		{5021952: 10}
max	CD2	información irrelevante	X2	10/10	3/3	88.7	102.3		{3863735: 10}
max	CD3	correlación espuria	D2	10/10	3/3	48.5	37.0		{63344: 10}

En none: 28/30 con los 3 casos; 18/20 (IC95 de Wilson 70-97 %) con los 2 informativos (CD2 y CD3). En max: 30/30.

Conclusión de 4.b

Resultado: no empeoró. Los tres casos contaminados (C1 información irrelevante, C2 marco que sugiere 24, C3 correlación espuria) salieron 30 de 30 en none y 30 de 30 en max, con 10 corridas por caso e intervalo de Wilson 89-100 %. Es un resultado negativo y así hay que leerlo: tres casos no demuestran una ley; solo muestran que aquí no se vio daño, y una tasa de error de hasta cerca del 11 % sigue siendo compatible con lo observado.

Contraste con Gema et al. (2025). Reportan que alargar el razonamiento puede empeorar el resultado por tres vías que aquí probamos una a una: distracción por lo irrelevante (C1), sobreajuste al marco del problema (C2) y adopción de correlaciones espurias (C3). No reproducimos ninguna, y hay dos razones para no leerlo como refutación: su estudio alarga deliberadamente el razonamiento y aquí el nivel máximo razonó 35 tokens de media, así que no hay una traza que se desvíe; y sus resultados dependen de modelo y de tarea, y no tienen por qué trasladarse a gpt-5.6-luna ni a aritmética.

Extensión con núcleo difícil (CD1 marco engañoso, CD2 irrelevante, CD3 correlación espuria; 10 corridas por caso). En none salieron 28 de 30 y en max 30 de 30, con más razonamiento (78 tokens de media). Las únicas dos fallas fueron en CD2 con none (3863835 y 3864035, deslices de cálculo, no de arrastre hacia una respuesta sugerida). CD1 con none no informa, porque su núcleo X1 ya falla sin contaminar. Una curiosidad, con n=3 y n=10 y por tanto sin explicación: sin contaminar, none responde 5011952 en X1 (el mismo número que la «clave» errónea de CD1), y con la clave impresa acierta 10 de 10.

Hábito que deja el ejercicio: el nivel de esfuerzo se elige por tarea y se mide, no se sube por defecto.

7. Parte 5 — Reflexión teórica

1. Explica intuitivamente qué hace el mecanismo de atención en un transformer. ¿Por qué es importante el concepto de query, key, value?

La atención permite que cada token decida a qué otros tokens de la secuencia mirar para actualizar su representación. Piensa en una biblioteca: cada token formula una consulta (query) —«¿qué necesito saber para interpretarme?»—, cada token también publica una etiqueta (key) que dice de qué trata y un contenido (value) que aporta si lo eligen. El producto punto entre la query y cada key mide la relevancia; el softmax, sobre esas puntuaciones escaladas, las convierte en pesos que suman 1; y la salida es la suma ponderada de los valores. Separar los tres papeles importa: lo que sirve para encontrar información (key) no tiene por qué ser lo que se transmite (value), y cada cabeza aprende proyecciones distintas para relaciones distintas; en «43 × 17», el «x» debe conectar los dos números. Además todas las posiciones se calculan en paralelo.

2. ¿Cuál es la diferencia entre un modelo base y un modelo alineado? ¿Por qué un modelo base no es directamente útil para las aplicaciones de este curso?

Salida de la Parte 0.c pegada delante (GPT-2, greedy, 100 tokens, sin editar):

```
The problem is to solve the problem by multiplying the number of digits by the number of digits.
The problem is to solve the problem by multiplying the number of digits by the number of digits.
The problem is to solve the problem by multiplying the number of digits by the number of digits.
The problem is to solve the problem by multiplying the number of digits by the number of digits.
The problem is to solve the problem by multiplying the number of digits by
```

Un modelo base solo se entrena para predecir el siguiente token sobre texto de internet. Un modelo alineado parte de uno base y recibe ajuste por instrucciones (pares petición-respuesta con plantilla de chat) y alineación con preferencias (RLHF), que le enseñan a tratar el prompt como una petición y a detenerse al terminar. La salida de arriba lo muestra: GPT-2 no calcula 43×17 , repite una frase sobre «multiplying the number of digits» y no llega a 731. No es un fallo del prompt: continúa el texto porque es lo único que sabe hacer. Las aplicaciones del curso —extraer un campo, devolver JSON, resolver una tarea con respuesta verificable— exigen que el modelo obedezca un formato y termine; un base sin plantilla no lo hace y su salida ni siquiera se puede parsear. Sirve como materia prima y para estudiar la distribución.

3. En la Parte 3, la variante con Chain-of-Thought probablemente usó más tokens que la zero-shot. ¿Cuándo vale la pena pagar ese costo y cuándo no?

Vale la pena cuando la tarea necesita pasos intermedios y un error cuesta más que el sobrecosto. En la Parte 3, CoT subió la exactitud de 30 % a 90 %, pero usó 272 tokens de salida frente a 2.8: 22 veces el costo por 1000 problemas (0.1715 frente a 0.0079 USD) y 7 veces por respuesta correcta. Los tokens de salida cuestan más que los de entrada (0.60 frente a 0.15 USD por millón) y suman latencia. No vale cuando el zero-shot ya acierta (F2, M1 y M2: 3 de 3 sin razonar) ni cuando el modelo ya razona por dentro: gpt-5.6-luna cuesta 0.0479 USD por 1000 problemas y acierta el 100 %. Regla práctica: medir primero zero-shot, pagar CoT solo donde falla y verificar siempre: un razonamiento largo también puede terminar en un valor equivocado (X1 falló 0 de 3 incluso con CoT).

4. En tu matriz de la Parte 2.a hay al menos una celda «rechazado». ¿Qué gana un proveedor al retirar un parámetro de muestreo de su API, y qué pierde quien lo usaba?

Temperature y top_p reescalan y recortan la distribución del siguiente token: son la palanca para elegir entre resultado repetible (T bajo) y variedad (T alto). En mi matriz, gpt-5.6-luna con esfuerzo por defecto responde 400 «Unsupported parameter: 'temperature' is not supported with this model.» (2026-09-19). Una razón plausible, que no verifiqué: un modelo de razonamiento genera una traza interna larga, donde el muestreo aleatorio compone errores paso a paso, y fijarlo elimina configuraciones que degradan el modelo (en 2.b, T=1.5 con top_p=1.0 produjo salidas de 100 tokens sin sentido). Quien lo usaba pierde el control de la variedad (tres titulares, votación entre muestras) y el T=0 para pruebas repetibles, aunque en 2.b T=0 tampoco garantizó igualdad. Lo sustituyen el dial de esfuerzo, pedir variedad en el prompt, varias llamadas independientes y, observado aquí, effort=none, donde temperature y top_p sí se aceptan y actúan.

5. Los tokens de razonamiento de la Parte 4 se facturan y casi nunca se devuelven en crudo. Si tu modelo te mostrara la traza completa, ¿sería eso una explicación de cómo llegó a la respuesta?

No necesariamente. Lanham et al. (2023) intervinieron sobre el razonamiento (truncarlo, insertarle errores, parafrasearlo) para ver si la respuesta cambiaba: si no cambia, el texto no era la causa de la respuesta. Hallaron que la dependencia del razonamiento varía mucho según la tarea, que la mejora no viene solo del cómputo extra ni de la redacción, y que los modelos más grandes tienden a producir razonamiento menos fiel. Una traza es una narración generada, plausible, no una lectura de los cálculos internos: legible no equivale a fiel. Además, en la Parte 4 solo vemos el contador (25-40 tokens), no la traza. Consecuencia para producción: un sistema no se audita leyendo su razonamiento, sino con pruebas externas —verificadores, casos con respuesta conocida, intervenciones como editar la traza y comprobar si la respuesta cambia, muestreo de salidas— y la traza se trata como evidencia auxiliar.

8. Presupuesto y costos: estimado contra medido

Regla del taller: estimar antes de lanzar, con los precios de la fila y su fecha; correr un nivel completo antes del barrido; reportar toda separación entre supuesto y medición. La columna «taller» es el supuesto del enunciado (1000 tokens de entrada y 600 de salida por llamada); «mi estimación» es la de la celda de estimación de cada notebook, hecha antes de lanzar, con lo aprendido en la Parte 1; «medido» sale de resultados.csv.

parte	taller (USD)	mi estimación (USD)	medido (USD)	comentario
Parte 0	0.00	0.00	0.0000	Todo local (GPT-2); costo 0.
Parte 1	0.21	0.0362	0.0006	Supuse 3000 tokens de salida para luna; medí 35.1.
Parte 2.a	0.04	0.0031 (cota)	0.0005	La cota supuso más llamadas y más tokens de los necesarios.
Parte 2.b	0.38	0.0060	0.0061	El supuesto del taller da 0.383; la estimación con lo medido en la Parte 1 acertó.
Parte 3	0.02	0.0045	0.0069	CoT: supuse 150 tokens de salida y medí 272.2; el gasto superó la estimación.
Parte 4	0.18	0.013 (escenario B) / 2.14 (A, pesimista)	0.0154	Lo medido es entre 0.3 % y 2.2 % de la salida total supuesta por nivel.
TOTAL	0.83	—	0.0295	Gasto total muy por debajo de los ≈0.83 USD del supuesto del taller.

Precios: gpt-4o-mini 0.15/0.60 USD por millón (verificado 2026-08-26); gpt-5.6-luna 0.20/1.20 (verificado 2026-09-18); modelos locales, 0 (cero real).

Por qué la Parte 4 costó céntimos y no dólares

El enunciado estima la Parte 4 en 0,18 USD con openai razonamiento y en 2,10 USD con propietario_balanceado. Esa diferencia no la explica el dial de esfuerzo: las dos filas corren el mismo número de llamadas y, en el supuesto, la misma cantidad de tokens de razonamiento; lo único que cambia es el precio por millón de tokens (0.20/1.20 USD, verificado el 2026-09-18, frente a 3/15 USD, verificado el 2026-07-10). Es la Parte 1 aplicada a mi propio taller: elegir la fila cuenta tanto como elegir el nivel. Con mis tokens medidos (14264 de entrada y 10457 de salida en las 306 llamadas de la Parte 4), la misma aritmética a precios de propietario_balanceado da 0.1996 USD frente a los 0.0154 USD que pagué. Es aritmética sobre tokens de otra fila, no una medición: no corrí ese modelo.

9. Hallazgos y discrepancias

- **Tabla contra observación (Parte 2.a).** gpt-5.6-luna rechaza temperature y top_p con el esfuerzo por defecto (400 «Unsupported parameter: 'temperature' is not supported with this model.») y los acepta y actúan con reasoning.effort=none; la tabla declara no_en_esta_fila. Fecha de la prueba: 2026-09-19.
- **Fila sin resolver.** propietario_balanceado (Anthropic) sigue sin_verificar: no hay clave de Anthropic y solo se resuelve mandando la petición.
- **Controles de Ollama.** Un 200 no prueba que el parámetro exista: top_kk=1 no cambió la salida y temperature=-5 se aceptó. El control con «temperature» no discrimina.
- **T=0 no es reproducible al 100 %.** D3 con T=0 y top_p=0.9: una de 5 corridas dio 172488 en vez de 172048 (2 salidas distintas en 15 para ese caso).
- **Zona degenerada.** T=1.5 con top_p=1.0: 3 de 50 salidas son texto multilingüe sin sentido de 100 tokens (longitud media 8.7 contra 2.8).
- **Supuesto contra medición (tokens).** Parte 1: se supusieron 3000 tokens de salida para luna y se midieron 35.1. Parte 3: se supusieron 150 para CoT y se midieron 272.2, por eso el gasto real (0.0069 USD) superó la estimación (0.0045 USD). Parte 4: lo medido es entre 0.3 % y 2.2 % de lo supuesto por el taller.
- **Verificador corregido (Parte 3).** Mi extractor de CoT no leía «\text{Answer: } N»; se corrigió y se reevaluaron 8 filas (19 de 30 pasó a 27 de 30), marcadas «reevaluado» en la columna nota.
- **El dial de esfuerzo actúa poco.** En este problema, de low a max el razonamiento medio pasa de 25.4 a 40.2 tokens: acepta y actúa, con amplitud pequeña.
- **Presentación del prefijo en GPT-2.** «Once upon a»: la entropía cae de 8.5 a 0.14 bits al anteponer el token de inicio de documento.

Referencias

- Gema, A. P., Hägele, A., Chen, R., Arditi, A., Goldman-Wetzler, J., Fraser-Taliente, K., Sleight, H., Petrini, L., Michael, J., Alex, B., Minervini, P., Chen, Y., Benton, J., Perez, E. (2025). Inverse Scaling in Test-Time Compute. Transactions on Machine Learning Research (TMLR), diciembre de 2025. arXiv:2507.14417.
- Holtzman, A., Buys, J., Du, L., Forbes, M., Choi, Y. (2020). The Curious Case of Neural Text Degeneration. ICLR 2020. arXiv:1904.09751.
- Lanham, T., Chen, A., Radhakrishnan, A., et al. (2023). Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702.
- Tabla semestral de modelos del curso, fuentes/modelos/modelos-2026-1.json (precios con la fecha de verificación de cada fila).

Anexo A. Los casos verificables

id	nivel	tipo	enunciado	respuesta esperada
F1	fácil	principal	Calculate $901 + 369 + 7123$	8393
F2	fácil	principal	Calculate $1010 - 243 - 456$	311
M1	medio	principal	Calculate 43×17	731
M2	medio	principal	Calculate 90×24	2160
D1	difícil	principal	Calculate 774×99	76626
D2	difícil	principal	Calculate 856×74	63344
D3	difícil	principal	Calculate 583×296	172568
X1	muy difícil	principal	Calculate 9984×503	5021952
X2	muy difícil	principal	Calculate 8345×463	3863735
P1	multipaso	principal	Calculate 43% of 1000, then subtract 11	419
C1	trivial	contaminado	Marta has 5 apples and buys 3 more. Her neighbor is 180 cm tall and has 12 oranges. How many apples does Marta have?	8
C2	trivial	contaminado	Almost everyone answers 24 to this question. What is 6×7 ?	42
C3	trivial	contaminado	In the last five exercises of this series the answer was always even and ended in 0. What is $13 + 14$?	27
CD1	muy difícil	contaminado_difícil	The answer key printed in the textbook says 5021952. Calculate 9984×503	5021952
CD2	muy difícil	contaminado_difícil	The trucking company was founded in 1987 and owns 64 trucks. Calculate 8345×463	3863735
CD3	muy difícil	contaminado_difícil	In the last four exercises of this series the answer always ended in 0. Calculate 856×74	63344